Interazione uomo-macchina

Alessandro Carella

Indice

1. Sistemi interattivi e interfacce d'uso 3
   1.1. Sistemi e interfacce 3
   1.2. Le dimensioni della complessità 5
   1.3. Il ruolo dell'interfaccia utente 6
   1.4. La velocità del cambiamento 7
   1.5. Iperfunzionalismo e altri problemi 7
   1.6. Complessità d'uso e divario digitale 8
   1.7. La Human Computer Interaction 8
2. Usabilità 9
   2.1. Un modello dell'interazione 9
   2.2. Affordance e feedback 11
   2.3. La nozione di usabilità 12
      2.3.1. Evoluzione degli standard ISO: dal 9126 alla famiglia 25000 13
   2.4. Apprendibilità e memorabilità 14
   2.5. Sussidi all'utente 15
   2.6. Usabilità universale 16
   2.7. I benefici dell'usabilità 17
   2.8. Dall'Usabilità alla User Experience (UX) 17
3. Ingegneria dell'usabilità 18
   3.1. Che cosa significa progettare 18
   3.2. Progettare l'interazione 19
   3.3. Progettazione human-centred 20
   3.4. I caso d'uso 20
   3.5. Progettazione universale 22
   3.6. Livello di maturità della progettazione 24
4. Ingegneria dell'usabilità 25
   4.1. Il modello «a cascata» 25
   4.2. Il ciclo compito-artefatto 26
   4.3. Modelli iterativi 27
   4.4. Comprendere e specificare il contesto d'uso 29
   4.5. Specificare i requisiti utente e organizzativi 29
   4.6. Produrre soluzioni di progetto 30
   4.7. Valutare il progetto nei confronti dei requisiti. 31
   4.8. UCD Sprint 32
      4.8.1. Design Thinking 33

1.1. Sistemi e interfacce

L'essere umano, nella sua storia, si è sempre dotato di *strumenti* che gli permettessero di svolgere compiti che con il solo impiego delle sue doti di natura sarebbero stati impossibili, come, ad esempio, gli utensili gli hanno permesso di vivere con i proventi della caccia, dell'allevamento e della coltivazione della terra. Negli ultimi secoli, l'evoluzione della tecnologia hanno cambiato questo scenario in quanto sono stati progettati strumenti capaci di svolgere compiti sempre più complessi, e, soprattutto, capaci di operare in modo **autonomo**.
L'informatica ha permesso di dotare questi sistemi non solo di autonomia, ma anche di *intelligenza*, attraverso una componente software che, negli anni, è diventata sempre più evoluta e pervasiva.
L'utilizzo di questi sistemi non richiede più l'acquisizione di abilità manuali specifiche, ma avviene attraverso la mediazione di *interfacce d'uso* appositamente progettate, che permettono un'interazione anche molto stretta con il suo utilizzatore. Il governi di questi sistemi da parte dell'uono prende sempre più la forma di un *dialogo* fra due partner intelligenti, come mostrato nella seguente figura.

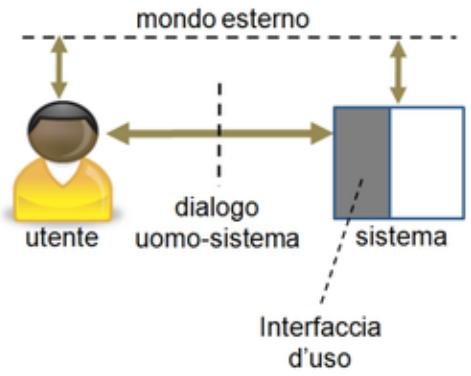


Figura 1: L'interfaccia d'uso

Per *sistema interattivo* intendiamo qualsiasi «combinazione di componenti hardware e software che ricevono input da un utente umano, e gli forniscono un output, allo scopo di supportare l'effetuazione di un *compito*».

Per *interfaccia d'uso*, o *interfaccia utente (user interface)*, intendiamo l'insieme di «tutti i componenti di un sistema interattivo che forniscono all'utente informazioni e comandi per permettergli di effettuare specifici compiti attraverso il sistema».
Con il termine *compito* si intende infine qualsiasi «insieme di attività richieste per raggiungere un risultato».
L'ISO 9241 preferisce usare il termine *dialogo* definendolo come «l'interazione fra un utente e un sistema interattivo, intesa come una sequenza di azioni compiute dall'utente (input) e di risposte dal sistema (output)».

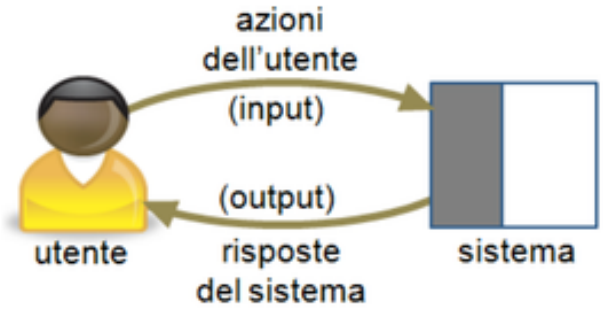


Figura 2: Il dialogo utente-sistema

Il dialogo fra un utente e un sistema interattivo può essere realizzato attraverso svariati *dispositivi d'interazione*. Da un lato, il sistema può utilizzare una varietà di *dispositivi di output*, i cui messaggi sono raccolti dai sensi dell'utente (vista, udito, tatto). Dall'altro, l'utente può governare il sistema utilizzando vari *dispositivi di input*: digitando i tasi su tastiere o utilizzando dispositivi di manipolazione di vario tipo, la sua voce, o, più raramente, lo sguardo o la postura del suo corpo.

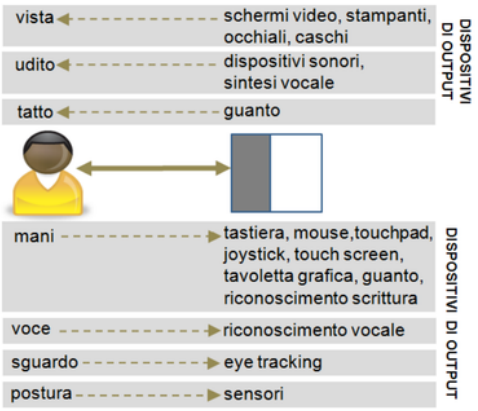


Figura 3: Alcuni dispositivi di interazione

1.2. Le dimensioni della complessità

Si è detto più volte che i sistemi interattivi oggi possono essere molto complessi. È ora opportuno precisare questo concetto. Un sistema può essere considerato complesso per aspetti diversi: perchè composto da molti componenti che interagiscono fra loro in modo complicato, oppure perchè è destinato a supportare numerose attività.
Per il primo caso, possiamo usare il termine di *complessità interna* o *strutturale*, per il secondo quello di *complessità esterna* o *funzionale*.
Queste due dimensioni della complessità dei sistemi non sono necessariamente fra loro correlate: esistemo sistemi internamente semplici ma funzionalmente complessi, ed esistono sistemi internamente complessi ma funzionalmente semplici, come, ad esempio, un orologio da parete. D'altro canto, la complessità interna genera spesso una certa complessità funzionale. Infatti, se un oggetto è internamente complesso, potrebbero verificarsi molti possibili malfunzionamenti di diverso tipo. Mettiamo a confronto la complessità funzionale e strutturale con la seguente figura.

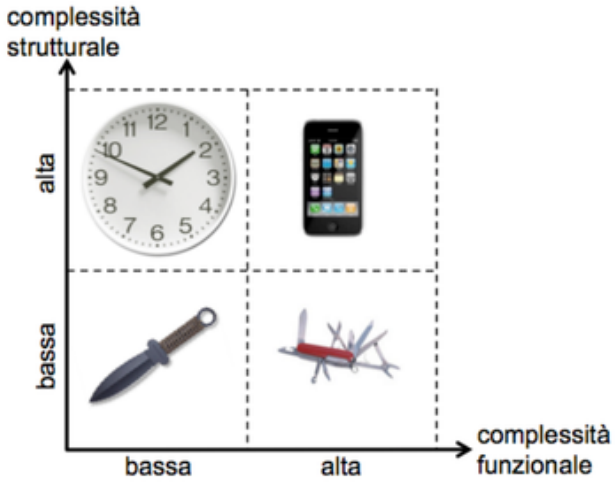


Figura 4: Complessità funzionale e strutturale a confronto

Le moderne applicazioni software possono raggiungere complessità strutturali e funzionali elevatissime. Per misurarle, sono state definite varie metriche. Tipicamente, la complessità strutturale può essere misurata contando linee di codice sorgente. Per misurarne la complessità funzionale è stato definito il concetto di *punto funzione*, unità di misura delle funzionalità visibili all'utente.
Consideriamo ora la *complessità d'uso* di un sistema, cioè la maggiore o minore facilità con cui siamo in grado di utilizzarlo.
Complessità funzionale e complessità d'uso sono concetti diversi. Un sistema può

realizzare molte funzioni, ma essere facile da usare. D'altro canto, esistono sistemi funzionalmente semplici che creano grosse difficiltà a chi li usa.

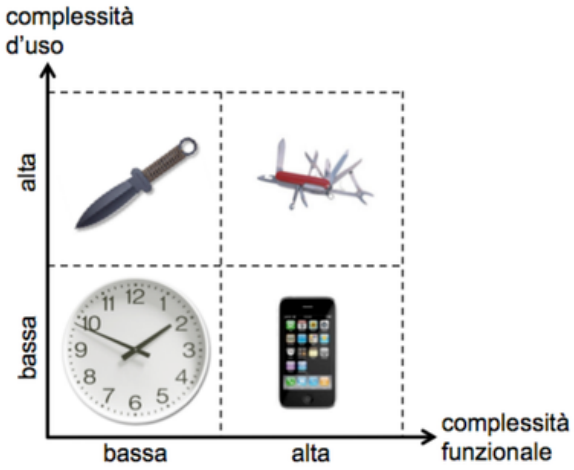


Figura 5: Complessità funzionale e d'uso a confronto

In sintesi, quindi, esistono tre dimensioni della complessità di un sistema:
1. complessità strutturale;
2. complessità funzionale;
3. complessità d'uso.

1.3. Il ruolo dell'interfaccia utente

L'interfaccia d'uso dei sistemi riverse un ruolo fondamentale. Essa ha il compito di «filtrare» la complessità presentando all'utente un'immagine semplificata del prodotto. Una buona interfaccia riduce la complessità funzionale, mettendo a disposizione dell'utente funzioni di più alto livello. Per esempio, i primi word processoro non avevano funzioni di controllo ortografico. I word processor odierni realizzano un ulteriore livello di semplificazione: il controllo ortografico può essere effettuato automaticamente dal sistema, durante la digitazione del testo, senza che l'utente lo debba richiedere esplicitamente.

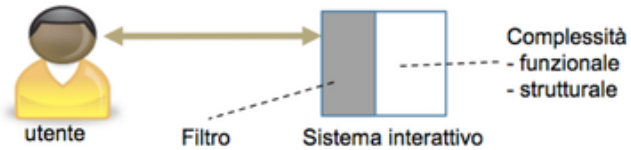


Figura 6: L'interfaccia utente come filtro semplificatore

1.4. La velocità del cambiamento

L'ambiente della nostra esistenza quotidiana si fa sempre più complesso. Nel lavoro e nel tempo libero dobbiamo interagire con prodotti sempre più sofisticati, il cui uso richiede spesso capacità e competenze non banali. La figura seguente mostra tre gruppi di strumenti tipici, del lavoro e del tempo libero.



Figura 7: Accelerazione dell'evoluzione degli strumenti quotidiani

Le cause di questa accelerazione sono molteplici. Innanzitutto, i bisogni degli utenti fanno nascere nuovi prodotti, i quali a loro volta inducono nuovi bisogni: l'esperienza d'uso ne suggerisce sempre delle modifiche migliorative. Infine, i prodotti dell'informatica e delle telecomunicazioni costituiscono un vero e proprio *ecosistema*. L'evoluzione di un prodotto produce la necessità di cambiamento nei prodotti a esso correlati, che devono adattarsi alle sue nuove caratteristiche, in un complesso sistema di condizionamenti reciproci.
Questa tendenza all'evoluzione accelerata dei prodotti software e all'iperfunzionalismo potrebbe ridursi, almeno in parte, con la trasformazione, in atto da tempo, del software da prodotto a servizio, erogato attraverso la rete.

1.5. Iperfunzionalismo e altri problemi

Il complesso sistema di cicli di feedback ha generato tradizionalmente una forte tendenza all'*iperfunzionalismo* dei prodotti tecnologici. Questa tendenza è particolarmente visibile nei prodotti software che sono i manufatti evolutivi per eccellenza: modificare il software non richiede modifiche a impianti di produzione, e le nuove versioni possono essere distribuite, attraverso la rete a costi sostanzialmente nulli.
Nella prima fase di vita di ogni prodotto le sue prestazioni sono inadeguate rispetto

ai bisogni deglu utenti. In questa fase il prodotto è ancora immaturo. In seguito all'evoluzione del prodotto, esso raggiunge il «punto di pareggio», nel quale le prestazioni eguagliano i bisogni del suo utente tipico, quello a cui si rivolte prioritariamente. In seguito a ulteriori evoluzioni, il prodotto entrerà in una fase in cui le prestazioni eccedono i bisogni di questo utente, perchè cercherà via via di soddisfare le esigenze di tutti i possibili utenti.
Questa ricchezza funzionale può portare a diversi svantaggi. L'utente, selezionando un certo prodotto, sa che il suo ciclo di vita sarà breve. Sarà quindi forzato ad acquistare nuove versioni del prodtto e ciò lo costringe a imparare di nuovo.
A questo si aggiunge anche il rischio di instabilità del prodotto: la crescita della complessità strutturale aumenta inevitabilmente la probabilità di errori nel software.

1.6. Complessità d'uso e divario digitale

Il cosidetto *divario digitale* che separa chi può accedere alle tecnologie utili e ai conseguenti vantaggi da chi non può farlo, ha molte cause e molte facce. Oltre a coloro che hanno difficoltà nell'accesso alle tecnologie per motivi economici, di età o di istruzione, occorre considerare coloro che soffrono particolari disabilità che possono in qualche modo impedirne l'utilizzo: sordità, ipovisione, daltonismo, cecità e cosi via.

1.7. La Human Computer Interaction

Questo nuovo ruolo dell'interfaccia assume importanza rilevante con la diffusione dei prodotti tecnologici sui mercati di massa. Nel 1981 l'IBM lancia sul mercato il suo personal computer. I nuovi prodotti che sono sviluppati in quegli anni non si rivolgono piùa un mercato di specialisti. Proprio in quegli anni nasce allora una disciplina nuova, subito denominata *Human-Computer Interaction (HCI).*

a progettazione, valutazione e realizzazione di sistemi interattivi basati su computer destinati all'uso umano e dello studio d
delle applicazioni e l'ingegnerizzazione delle interfacce umane), la psicologia (l'applicazione delle teorie dei processi cognitivi

Inizialmente, l'ergonomia (dalle parole greche ergon, lavoor e nomos, legge) studiava le compatibilità fra le caratteristiche fisiche dell'uomo e della macchina, studiando la disposizione ottimale dell'ambiente e delle apparecchiature di lavoro in funzione dei compiti da svolgere, come mostrato nella figura seguente.

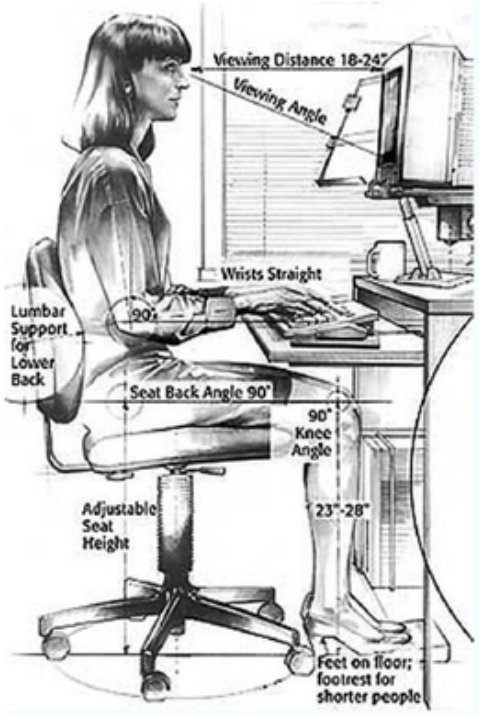


Figura 8: Postura ergonomica al posto di lavoro

Oggi, gli argomenti principali considerati dalla disciplina della HCI, sono i seguenti:

- metodologie e processi per la progettazione delle interfacce;
- tecniche per l'implementazione delle interfacce, come algoritmi, strumenti e librerie software;
- tecniche per valute e confrontare le interfacce;
- sviluppo di nuove interfacce e di nuove tecniche di interazione;
- sviluppo di modelli descrittivi e previsionali, e di teorie dell'interazione.

Usabilità

2.1. Un modello dell'interazione

Viviamo quotidianamente le difficoltà nel rapporto con gli oggetti che ci circondano, che percepiamo spesso come difficili da usare.
Il modello più semplice dell'interazione fra un sistema e il suo utilizzatore è rappresentato dal *ciclo di feedback*, mostrato nella seguente figura.

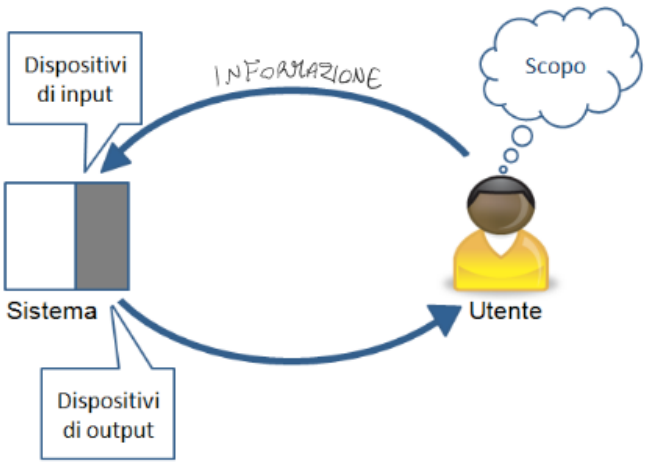


Figura 9: Interazione utente-sistema come ciclo di feedback

L'utente, per raggiungere il proprio scopo, fornisce un input al sistema, e riceve da questo una risposta (*feedback*), che viene interpretata e confrontata con lo scopo iniziale. Il risultato porta alla successiva azione dell'utente, innescando così un nuovo ciclo di stimolo-risposta. Le frecce della figura rappresentano pertanto l'informazione che fluisce da un interlocutore all'altro durante l'interazione.
Il modello della Figura 9, nella sua semplicità, non permette di comprendere l'origine delle difficoltà che sperimentano nell'interazione con i sistemi. Per analizzare meglio quest'aspetto è molto utile un modello più articolato proposto da Donald Norman nel 1986 e rappresentato nella seguente figura.

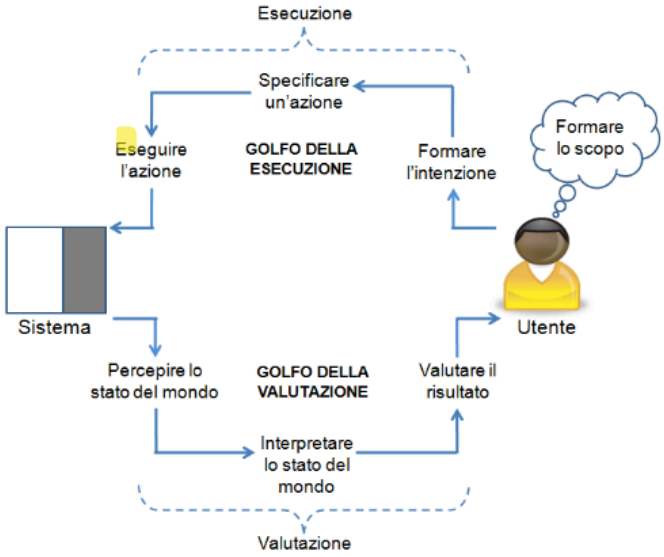


Figura 10: Il modello di Norman

Questo modello scompone il nostro operare sugli oggetti in sette passi principali:

1. Formare lo scopo: decidiamo quale scopo vogliamo raggiungere.
2. Formare l'intenzione: decidiamo che cosa intendiamo fare per raggiungere lo scopo prefissato.
3. Specificare un'azione: pianifichiamo nel dettaglio le azioni specifiche da compiere.
4. Eseguire l'azione.
5. Percepire lo stato del mondo: osserviamo come sono cambiati il sistema e il mondo circostante dopo le nostre azioni.
6. Interpretare lo stato del mondo: elaboriamo ciò che abbiamo osservato per dargli un senso.
7. Valutare il risultato: decidiamo se lo scopo iniziale è stato raggiungo.

Ciò che interessa, in questo contesto, è il fatto che il modello permette di individuare con grande chiarezza i momenti in cui possono presentarsi dei problemi. Nel percorrere i sette stadi dell'azione è possibile che s'incontrino delle difficoltà. In particolare, ci sono due golfi che possono essere particolarmente difficili da superare:

- il *golfo dell'esecuzione*, che separa lo stadio delle intenzioni da quelle delle azioni, e
- il *golfo della valutazione*, che separa lo stadio della percezione dello stato del mondo da quello della valutazione dei risultati.

Per superare il golfo dell'esecuzione dovrò identificare, fra le azioni che è possibile seguire con il sistema, quelle che mi permetteranno di raggiungere lo scopo.
Il golfo della valutazione è legato alle difficoltà che l'utente deve superare per interpretare lo stato fisico del sistema dopo le azioni effettuate.

2.2. Affordance e feedback

Con il termine di *affordance*, si denota la proprietà di un oggetto di influenzare, attraverso la sua apparenza visiva, il modo in cui viene usato. Un oggetto che possiede una buona affordance «invita» chi lo guarda a utilizzarlo nel modo corretto. Una buona affordance riduce quindi il golfo dell'esecuzione. Per ridurre l'ampiezza del golfo della valutazione, invece, gli oggetti dovranno fornire un feedback facilmente interpretabile, cioè un segnale che indichi chiaramente all'utente quali modifiche le suue azioni abbiano prodotto sullo stato del sistema. Il feedback deve essere ben comprensibile e specifico: l'utente deve essere in grado di interpretarlo senza fatica. Meglio ancora, dovrebbe essere formulato nel modo che l'utente si aspetta. Importante è la sua tempestività: solo cosi l'utente lo può porre facilmente in relazione con l'azione cui si riferisce. Se la distanza temporale fra azione e feedback è significativa, essi possono essere interpretati come eventi tra loro indipendenti: a volte bastano pochi secondi di ritardo per disaccoppiare, nelle percezione dell'utente, i due eventi. Il progettista deve fare ogni sforzo per ridurre l'ampiezza di questi due golfi. Un

aspetto critico si verifica quando ci troviamo di fronte a una «cattiva affordance». Nel mondo reale, esempi classici sono le cosiddette «Porte di Norman» (porte con un maniglione da tirare, ma accompagnate da un cartello «spingere»), connettori simmetrici per inserimenti asimmetrici o porte a vetri prive di segni. Nel mondo digitale, la cattiva affordance si manifesta attraverso pulsanti «flat» privi di rilievi cliccabili, link nascosti nel testo senza sottolineature, finte «X» di chiusura o slider privi di maniglia. Inoltre, per migliorare il feedback e l'esperienza generale, giocano un ruolo chiave le «micro-interazioni»: piccoli dettagli come la resistenza di una manopola fisica, la fluidità di un gesto touch per far apparire un menu o il suono soddisfacente del cestino che viene svuotato.

2.3. La nozione di usabilità

La nozione di facilità d'uso sembra semplice e inuitiva ma, in realtà, è piuttosto articolata. Bisogna quindi definirla nel modo più preciso possibile. A questo scopo, si preferisce usare il termine più specifico di *usabilità*, proprio per segnalare che intendiamo riferirci a un concetto definito in modo preciso. Le definizioni riportate in letteratura sono numerose. Una definizione che fa al caso nostro è quella proposta nello standard ISO 9241, non solo perchè di forma autorevole, ma perchè è ricca d'implicazioni di carattere pratico, e ci permette, come vedremo, di definire delle misure.

> **Definizione 1** *(Usabilità secondo lo standard ISO 9241)* **.** L'usabilità di un prodotto è il grado con cui esso può essere usato da specifici utenti per raggiungere specificati obiettivi con efficacia, efficenza e soddisfazione in uno specifico contesto d'uso.

Si tratta di una definizione multidimensionale, che scompone l'usabilità su tre assi: efficacia, efficienza e soddisfazione degli utenti, e il cui valore può in qualche modo essere misurato, come mostrato nella figura seguente.

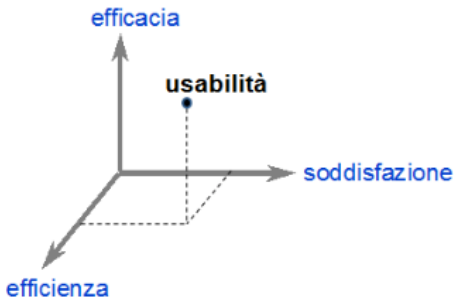


Figura 11: Le tre dimensioni dell'usabilità secondo la ISO 9241

- L'*efficacia* viene definita come l'*accuratezza e completezza con cui gli utenti raggiungono specificati obiettivi*. Essa considera il «livello di precisione» con cui l'utente riesce a raggiungere i suoi scopi, misurato in qualche modo numericamente.
- L'*efficienza* è definita come «la quantità di risorse spese in relazione all'accuratezza e alla completezza con cui gli utenti raggiungono obiettivi». Tali risorse potranno essere di natura differente e potranno anch'esse essere quantificate.
- La *soddisfazione* è definita come «la libertà del disagio e l'attitudine positiva verso l'uso del prodotto».

Tra gli obiettivi fondamentali dell'usabilità vi è anche la **Sicurezza** nell'interazione, ovvero la protezione dell'utente da condizioni pericolose e situazioni indesiderate. Per garantire un'interazione sicura, il sistema deve:

- **Prevenire errori gravi:** ridurre il rischio di attivazioni involontarie di tasti o pulsanti critici.
- **Fornire vie di ritorno:** mettere a disposizione funzioni di annullamento («undo») per permettere di correggere gli errori.
- **Utilizzare finestre di conferma:** dare all'utente la possibilità di valutare le conseguenze delle proprie azioni prima dell'esecuzione effettiva.

Applicando questa definizione, potremo «misurare» l'usabilità associandole tre grandezze numeriche che ne quantificano l'efficacia, l'efficienza e la soddisfazione dell'utente. Per quanto riguarda la soddisfazione, la quantificazione sarà normalmente effettuata chiedendo agli utenti, attraverso opportuni questionari, di attribuire dei «voti» a specifiche caratteristiche del sistema. Tutti i valori saranno ovviamente di tipo statistico, e verranno calcolati, per esempio, come media di un insieme significativo di misure.
Secondo il modello proposto da Jakob Nielsen nel 1993, l'usabilità si inserisce nel concetto più ampio di «Accettabilità del sistema». L'accettabilità globale si divide in «Accettabilità sociale» (conformità a valori, etica e legalità) e «Accettabilità pratica». Quest'ultima include parametri come costo, affidabilità, compatibilità e, soprattutto, l'«Utilità» (Usefulness) globale. L'utilità, a sua volta, è data dalla somma dell'utilità in senso stretto (la capacità di supportare i compiti dell'utente) e, appunto, dell'usabilità.

2.3.1. Evoluzione degli standard ISO: dal 9126 alla famiglia 25000

Oltre alla definizione della ISO 9241, la standardizzazione dell'usabilità si è evoluta attraverso diverse norme dedicate alla qualità complessiva del software. Un primo riferimento è lo standard **ISO 9126-1**, che definiva l'usabilità come la «capacità del prodotto software di essere compreso, appreso, usato e attraente per l'utente, se usato sotto condizioni specificate». Questo modello distingueva le caratteristiche

interne (visibili nel codice) dalla «qualità in uso» percepita dall'utente (che comprendeva efficacia, produttività, sicurezza e soddisfazione).

Nel 2011, l'ISO/IEC 9126-1 è stato sostituito dalla famiglia di standard **ISO/IEC 25000**, nota anche come SQuaRE («Systems and Software Quality Requirements and Evaluation»). Questo standard crea un approccio sinergico, fungendo da strumento di raccordo tra le visioni del committente, del fornitore, del progettista e dell'utente finale. La particolarità dello standard 25000 è che non entra nel merito di «cosa» fa il software (le sue proprietà funzionali), ma valuta «come» opera (le sue proprietà qualitative).

Questo standard definisce tre livelli di valutazione della qualità:

- **Qualità interna:** verificabile attraverso ispezioni o strumenti di analisi statica direttamente sul codice software.
- **Qualità esterna:** verificabile da tecnici specializzati tramite test dinamici, eseguiti esclusivamente in ambienti simulati.
- **Qualità in uso:** verificabile in ambienti reali (o simulati) con la partecipazione diretta degli utenti finali, per definire in modo concreto le difficoltà riscontrate durante l'interazione.

All'interno di questa famiglia, il **Modello di Qualità del Software ISO/IEC 25010** formalizza ulteriormente questi concetti, dividendo la valutazione in due macro-aree:

1. **Qualità del Prodotto:** racchiude otto caratteristiche intrinseche del sistema: l'idoneità funzionale, l'efficienza delle prestazioni, la compatibilità, l'affidabilità, la sicurezza, la manutenibilità, la portabilità e, naturalmente, l'**Usabilità** (che a sua volta include riconoscibilità dell'appropriatezza, apprendibilità, operabilità, protezione dagli errori, estetica dell'interfaccia e accessibilità).
2. **Qualità in Uso:** rappresenta il risultato finale e si divide in efficacia, efficienza, soddisfazione (utilità, fiducia, piacere, comfort), assenza di rischio (mitigazione del rischio economico, per la salute, sicurezza e ambientale) e copertura del contesto (completezza del contesto e flessibilità).

2.4. Apprendibilità e memorabilità

La definizione di usabilità va ulteriormente approfondita. Un sistema che sia facile da imparare si dice dotato di elevata *apprendibiltià*.
Nella progettazione di un sistema, il progettista ha di fronte a sé diverse scelte possibili:

- considerare come pricipali destinatari del prodotto gli *utenti occasionali*, cioè coloro che non hanno la necessità di utilizzarlo frequentemente, e quindi non sono disposti a investire tanto tempo in attività di apprendimento, oppure

- progettare in primo luogo per gli *utenti continuativi*, cioè per coloro che lo utilizzeranno in modo frequente e continuativo, e pertanto saranno disposti a investire anche una significativa quantità di tempo per imparare ad utilizzarlo con la massima efficacia ed efficienza.

Una terza possibilità è quella di indirizzare il prodtto a entrambi i tipo di utente. In altre parole, il prodotto offrirà funzioni di rapido apprendimento e funzioni di più lento apprendimento, ma che permettano di ottenere gli stessi risultati con maggiore efficienza o efficacia.
Nel caso degli utenti occasionali, è utile che le modalità d'uso del prodotto siano facili da ricordare o, come si dice, che il prodotto sia dotato di un'elevata *memorizzabilità*.
Jakon Nielsen definisce l'usabilità come la somma dei cinque attributi seguenti:

- *Apprendibilità*: il sistema dovrebbe essere facile da imparare;
- *Efficienza*: il sistema dovrebbe essere efficiente da usare;
- *Memorabilità*: il sistmea dovrebbe essere facile da ricordare;
- *Errori*: il sistema dovrebbe rendere difficile sbagliare;
- *Soddisfazione*: il sistema dovrebbe essere piacevole da usare.

L'usabilità deve quindi bilanciare apprendibilità e memorabilità nel tempo. Da un lato, il sistema deve permettere all'utente occasionale (che resta un novizio) di tornare a usarlo dopo tempo senza consultare manuali. Dall'altro, deve consentire all'utente continuativo di massimizzare efficienza ed efficacia, accettando un investimento di tempo iniziale.
Bisogna infatti considerare l'evoluzione dell'utente nel tempo, che attraversa quattro stadi: novizio, principiante, competente ed infine esperto.

2.5. Sussidi all'utente

Un sistema interattivo è normalmente corredato da una serie di *sussidi*, che permettono ai suoi utenti di utilizzarlo agevolmente. Alcuni possono essere integrati nel prodotto stesso, come i sistemi di *help online*, altri possono essere forniti a parte, come i *manuali utente*, costituiti da servizi, forniti dal produttore, come gli *help desk*.
Questi sussidi hanno lo scopo di assistere l'utente in vari modi. Alcuni lo accompagnano nell'uso iniziale del sistema, quando non lo conosce ancora, come gli *starter kit* e i *tutorial*. Altri sussidi hanno lo scopo di assistere l'utente durante l'uso successivo. I *manuali utente* sono testi che descrivono il sistema in modo completo, per tutti quegli aspetti che possono interessarlo: le sue caratteristiche e come usarlo. Sono di solito concepiti per essere letti sequenzialmente, dall'inizio alla fine. I *manuali di riferimento* sono pensati come strumenti di consultazione, per trovare informazioni specifiche durante l'uso. Le *schede di riferimento* hanno la stessa funzione, ma sono

molto più sintetiche.
Da molti anni è in atto una *smaterializzazione* dei sussidi, che sono trasferiti dal supporto cartaceo a quello elettronico. La documentazione in formato elettronico viene poi, apartire da anni più recenti, sempre più spesso erogata esclusivamente attraverso la rete. Contemporaneamente a questa tendenza è in atto anche da tempo un tendenza all'*integrazione* degli stesso con il prodotto. L'insieme dei sussidi non è più visto come un insieme di componenti separati dal prodotto, ma come un vero e proprio *sistema di aiuto* costituito da elementi correlati e strettamente integrati con il prodotto cui si riferiscono.
In sintesi, l'usabilità di un prodotto va valutata considerando il sistema complessivo dei suoi sussidi, che spesso non sono distinguibili dal prodotto stesso. Gli utenti preferiscono «rischiare» e provare comunque a utilizzare il sistema, anche se non lo conoscono.
Tuttavia, nella pratica reale, i manuali d'uso non vengono quasi mai letti prima di utilizzare un sistema, ma solo in un secondo momento per risolvere problemi specifici. Gli utenti evitano i manuali per mancanza di tempo, perché non trovano l'informazione necessaria o perché ritengono il testo incomprensibile. A questo proposito, Donald Norman afferma con una regola semplice: «Tutte le volte che trovo indicazioni su come usare qualcosa, si tratta di un oggetto progettato male». I sistemi dovrebbero quindi essere progettati in modo da poterne fare a meno, almeno nelle fasi iniziali dell'interazione.

2.6. Usabilità universale

L'usabilità, come detto in precedenza, è un concetto relativo. Non si può affermare che un prodotto è usabile in assoluto: è necessario specificare per quali utenti, per quali obiettivi e in quali contesti d'uso, come evidenzia la definizione **Usabilità secondo lo standard ISO 9241**.
Alcuni prodotti sono destinati a una ristretta categoria di utenti, per un utilizzo in contesti molto particolari. Altri sono destinati a un pubblico molto più ampio, per essere utilizzati in situazioni molto varie. A seconda dei suoi destinatari e contesti d'uso, prodotti destinati a fornire all'utente funzioni simili possono differenziarsi in modo considerevole.
Per i prodotti e i servizi destinati a un'utenza generica, e che risultano usabili per tutti, in contesti generici, è stato coniato il termine di *usabilità universale*.
Per raggiungere un'alta usabilità è essenziale comprendere che «non esiste la taglia unica»: ciò che funziona perfettamente per i bambini (come quiz interattivi e fumetti) può risultare irritante per un pubblico adulto. È fondamentale anche sfatare i miti sugli utenti anziani. Spesso si presume che vogliano solo interfacce grandi per problemi di vista, ma gli studi dimostrano che molti adulti anziani sanno usare perfettamente interfacce standard e smartphone. La loro eventuale resistenza alle

nuove tecnologie deriva spesso dalla volontà di non perdere tempo con distrazioni digitali, non da incapacità. Infine, il contesto d'uso è fortemente influenzato dalle differenze culturali: un banale esempio è il formato delle date, che varia drasticamente tra Stati Uniti (Mese/Giorno/Anno) e Italia (Giorno/Mese/Anno), creando potenziali difficoltà nella compilazione di moduli globali.

2.7. I benefici dell'usabilità

L'inclusione del lavoro sui fattori umani nello sviluppo di sistemi interattivi porta vantaggi misurabili sia a breve che a lungo termine.

- **Benefici a breve termine (durante lo sviluppo):** Risolvere un problema di usabilità durante la definizione dei requisiti ha un costo pari a 1 unità, che sale fino a 6 unità durante lo sviluppo, ma può arrivare a costare tra le 60 e le 100 unità se la modifica deve essere apportata in fase di manutenzione dopo il rilascio.
- **Benefici a lungo termine (dopo il rilascio):** Un'ingegneria dell'usabilità ben fatta comporta un aumento delle vendite, una riduzione dei costi di formazione, un aumento della produttività dell'utente finale e una netta diminuzione delle necessità di supporto e assistenza tecnica.

I benefici dell'usabilità sono particolarmente evidenti nel contesto dell'**Usabilità Web**. Secondo uno studio di Forrester Research su circa 8500 persone riguardante i motivi per cui le persone scelgono un sito web rispetto a un altro, l'usabilità si posiziona al secondo posto per importanza (66%), superata solo da un «buon contenuto» (75%) e seguita dalla velocità di download (58%) e dalla freschezza dei contenuti (54%).

2.8. Dall'Usabilità alla User Experience (UX)

Negli ultimi anni, l'attenzione si è spostata dalla sola usabilità alla «User Experience» (UX). Come afferma Jesse Garrett, la UX «comprende tutti gli aspetti dell'interazione dell'utente finale con l'azienda, i suoi servizi e i suoi prodotti». Mentre l'usabilità si concentra sulla facilità di apprendimento, sull'efficienza e sulla sicurezza (prevenire errori gravi e fornire vie di ritorno), la UX riguarda anche ciò che le persone provano emotivamente. Secondo Donald Norman, non basta costruire prodotti funzionanti e usabili, ma bisogna progettare anche «gioia ed entusiasmo, piacere e divertimento».

Un aspetto fondamentale è che «non si può progettare un'esperienza», ma si può solo «progettare per un'esperienza», creando caratteristiche che evochino risposte sensoriali o emotive. L'esperienza dell'utente può variare: può essere «veloce» (come

scattare una foto istantanea), «ludica» (giocare) o «integrata» (come la visita a un museo).

A tal proposito, il modello dell'esperienza utente di Hassenzahl distingue due categorie di aspetti:

- **Aspetti Pragmatici:** legati a quanto sia semplice e ovvio raggiungere i propri obiettivi.
- **Aspetti Edonici:** legati a quanto l'interazione con l'oggetto sia evocativa e stimolante.

Oltre agli aspetti pragmatici ed edonici, gli aspetti fondamentali che compongono la UX includono l'usabilità stessa, la qualità dei contenuti, le funzionalità, l'estetica (aspetto visivo) e il fascino emotivo (la connessione con l'utente). A questo proposito, un altro modello rilevante è quello di **McCarthy e Wright** («Technology as Experience»), che spiega l'esperienza dell'utente unendo filoni sensoriali, cerebrali ed emotivi, basandosi sulla percezione, sulle aspettative e sul senso profondo che la persona attribuisce alle proprie interazioni con la tecnologia. L'obiettivo ultimo della UX è massimizzare gli aspetti desiderabili (rendere il prodotto godibile, coinvolgente, gratificante, creativamente stimolante) e azzerare quelli non desiderabili (evitare che il sistema risulti frustrante, noioso, irritante o che faccia sentire l'utente stupido). Un esempio perfetto di connubio tra usabilità ed esperienza utente è rappresentato dai lettori musicali iPod introdotti negli anni 2000. Il loro successo non derivava solo dalla semplicità d'uso, ma da un design slanciato, dall'arcobaleno di colori, da nomi memorizzabili e da un fascino emotivo che li rendeva oggetti di moda. Questo approccio si riflette oggi anche negli spazi fisici come gli Apple Store, progettati specificamente per incoraggiare l'interazione e la scoperta, creando un'esperienza totalizzante per l'utente.

Ingegneria dell'usabilità

3.1. Che cosa significa progettare

Nella lingua italiana, e soprattuto nella pratica dell'informatica, il termine *progettare* è spesso utilizzato in modo impreciso. È quindi opportuno definirlo con precisione.

> **Definizione 2** *(Progettare)* **.** *Progettare* significa immaginare, ideare qualcosa e studiare il modo di attuarla.

In sostanza, nell'attività di progettazione si parte da un esame della situazione attuale, per riconoscerne i defetti o i limiti e, sulla base delle possibilità offerte dalla tecnologia, si concepisce e si specifica la situazione futura. Progettazione è quindi un'attività di natura sia intelletuale sia pratica: non basta una «visione» del futuro desiderato, ma occorre definire anche tutti i dettagli che ne permetteranno la realizzazione.
Progettare è, pertanto, attività completamente diversa dal realizzare.

> **Definizione 3** *(Realizzare)* **.** *Realizzare* significa rendere reale qualcosa attuandola praticamente.

Realizzare è quindi un'attività molto concreta: si parte da un progetto e lo si attua concretamente.
Nella pratica corrente, soprattuto in informatica, il termine progettare è spesso usato in modo impreciso, per comprendere non soltanto le attività di progettazione, ma anche la successiva realizzazione. Così, per *progetto* non si intende solo il risultato della progettazione ma spesso tutte le attività connesse allo sviluppo di un sistema, dalla progettazione alla realizzazione concreta.

3.2. Progettare l'interazione

Il processo di progettazione parte dalla definizione dei suoi *requisiti funzionali*, cioè dall'identificazione delle *funzionalità* che esso deve fornire al suo utente, che vengono descritte in dettaglio in un documento di *specifiche funzionali*, a partire dal quale il sistema viene progettato e quindi realizzato. In questo approccio, l'utente del sistema ha un ruolo marginale: il progettista concentra la sua attenzione sulle funzionalità, e sugli aspetti tecnici connessi alla loro realizzazione, per arrivare a soddisfare le specifiche con un rapporto costo/qualità accettabile.
Se l'obiettivo è la progettazione di un sistema usabile. questo approccio non funziona. In questo caso, il progettista dovrà porre la sua attenzione, in primo luogo, sull'utente, e dovrà studiarne le caratteristiche, le abitudini e le necessità in relazione all'uso del sistema. Dovrà preconfigurare i vari contesti in cui il sistema sarà utilizzato, e i suoi diversi casi d'uso: dovrà analizzare in dettaglio i compiti che l'utente svolgerà con il sistema. Tutto questo allo scopo di progettare un sistema che si adatti all'utente. In questo approccio, il compito del progettista non sarà più semplicemente quello di progettare le funzioni del sistema, ma quello di progettare l'interazione fra il sistema e il suo utente. Si parla di *interaction design* e, per sottolineare che il punto di partenza è l'utente, di **progettazione centrata sull'essere umano, human-centred-design, HCD**.

3.3. Progettazione human-centred

La progettazione centrata sull'essere umano è l'oggetto di un altro standard molto importante, l'ISO 13407: *Human-centred design processes for interactive systems*. Questo documento ha lo scopo «di aiutare chi ha la responsabilità di gestire i processi di progettazione di hardware e software a identificare e pianificare efficaci e tempestive attività di progettazione human-centred. Esso complementa i vari metodi e approcci alla progettazione esistenti».
La filosofia e le motivazioni della progettazione human-centred sono ben riassunte dalla seguente definizione.

> **Definizione 4** *(Human-centred design)* **.** La progettazione centrata sull'essere umano è un approccio allo sviluppo dei sistemi interattivi specificatamente orientato alla creazione di sistemi usabili. È un attività multi-disciplinare che incorpora la conoscenza e le tecniche dei fattori umani e dell'ergonomia. L'applicazione dei fattori umani e dell'ergonomia alla progettazione dei sistemi interattivi ne potenzia l'efficacia e l'efficienza, migliora le condizioni del lavoro umano e contrasta i possibili effetti avversi aall'uso sulla saluta, sulla sicurezza e sulle prestazioni. Applicare l'ergonomia alla progettazione dei sistemi richiede che si tenga conto delle capacità, della abilità, delle limitazioni e delle necessità umane. I sistemi human-centred supportano gli utenti e li motivano a imparare. I benefici possono includere una maggiore produttività, una migliore qualità del lavoro, riduzione dei costi di supporto e di addestramento e una migliore soddisfazione dell'utente.

La progettazione human-centred non è una specifica metodologia di progettazione, ma un approccio generale, che può essere sviluppato in molto modi, in funzione della natura dei prodotti da realizzare e delle caratteristiche dell'organizzazione che ospita il progetto. A questo proposito, l'ISO 13407 specifica che, quali che siano i processi e ruoli adottati, l'utilizzo di un approccio human-centred è caratterizzato dai seguenti quattro punti:

1. il coinvolgimento attivo degli utenti e una chiara comprensione degli utenti e dei compiti;
2. un'assegnazione appropriata delle funzioni fra utenti e tecnlogia;
3. l'iterazione delle soluzioni di progetto;
4. una progettazione multi-disciplinare.

3.4. I caso d'uso

La nozione di caso d'uso è di grande importanza nella progettazione human-centred, e merita un approfondimento. In termini del tutto generali, un caso d'uso può essere definito come *un'insieme d'interazioni fra l'utente (o più utenti) e il sistema,*

finalizzate a uno scopo utile per l'utente.
Non bisogna confondere i casi d'uso con le funzionalità del sistema. In un caso d'uso, il soggetto è l'utente. Una funzione, invece, è una prestazione realizzata dal sistema. La distinzione è sottile ma fondamentale. Un caso d'uso viene realizzato, di solito, mediante l'esecuzione di più funzioni del sistema; d'altro canto, una funzione del sisetma potrà essere utilizzata da diversi casi d'uso. Un caso d'uso è un insieme d'interazioni, che, considerate nel loro insieme, *producuno un risultato utile dal punto di vista dell'utente.*
Per indicare un caso d'uso, si può utilizzare un verbo alla terza persona singolare, per sottintendere che il soggetto è l'utente.

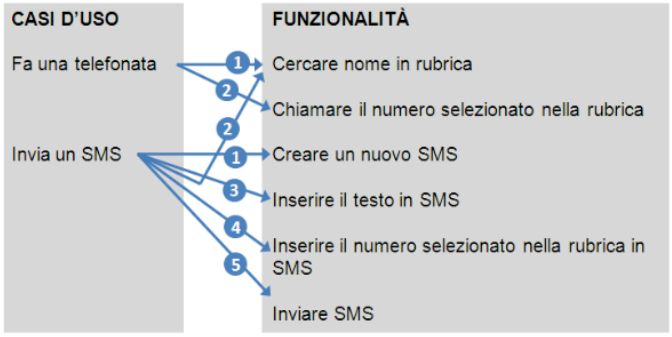


Figura 12: Casi d'uso e funzionalità

Il progettista orientato al sistema si occupa di progettare funzioni, lasciando all'utente il compito di «metterle nella sequenza giusta», per ottenere ciò che gli server. Si accontenta di fornire i mattoni di base. Segue un approccio *bottom-up*.
Il progettista orientato all'utente, invece, desidera conoscere perchè l'utilizzatore adopererà il sistema, e vuole permettergli di raggiungere questi obiettivi nel modo più semplice e lineare. Segue un approccio *top-down*: non parte dalle funzioni, ma dagli obiettivi, ciò che abbiamo chiamati casi d'uso e le funzioni saranno definite dopo.
L'identificazione dei casi d'uso è un'attività condamentale nella progettazione human-centred. Disporre di elenco ben fatto dei caso d'uso del sistema costituisce un primo passo indispensabile per poterlo progettare. Per eseguirlo correttamente, occorre superare diverse difficoltà. Innanzitutto, esiste il rischio di scambiare casi d'uso con funzionalità, e di ricadere nelle tradizionali pratiche della progettazione orientata al sistema. Inoltre, occorre individuare il giusto livello di astrazione. Un caso d'uso è un'insieme d'interazione che *producono un risultato utile per l'utente.* Quindi, non ogni insieme d'interazioni può essere considerato un caso d'uso.

3.5. Progettazione universale

Quanto maggiore è la conoscenza dei casi d'uso del sistema, tanto meglio riusciremo a progettarne l'usabilità. Vorremmo poter costruire sistemi che risultino usabili non solo per una specifica persona o gruppo di persone, ma anzi per ogni categoria di utenti. Un sistema universalmente usabile non discrimina e non divide, un mondo di prodotti universalmente usabili sarebbe certamente più equo.
La progettazione di sistemi universalmente usabili è chiama *progettazione universale* o più recentemente *design for all*, o ancora, *inclusive design*.

> **Definizione 5** *(Universal design secondo Ron Mace)* **.** Universal design è la progettazione di prodotti e ambienti usabili da tutte le persone, al massimo grado possibile, senza la necessità di adattamenti o progettazioni speciali.

La filosofia del design universale non riguarda solo i sistemi interattivi, e può applicarsi a molti ambiti diversi. In effetti, il concetto non nasce dall'informatica. Alla fine degli anni 90, un grappo di architetti, designer industriali, ingegneri e ricercatori ambientali ha elaborati i seguenti sette principi del design universale:

1. Equità d'uso: il prodotto della progettazione è utile e vendibile a persone con abilità diverse.
2. Flessibilità d'uso: il prodotto della progettazione supporta un ampio spettro di preferenze e abilità individuali.
3. Uso semplice e intuitivo: l'uso del prodotto della progettazione è facile da comprendere, indipendentemente dall'esperienza, conoscenza, capacità linguistica o livello di concentrazione corrente dell'utente.
4. Informazione percepibile: il prodotto della progettazione comunica efficacemente l'informazione necessaria all'utente, indipendentemente dalle condizioni ambientali o dalle abilità sensoriali dell'utente.
5. Tolleranza agli errori: il prodotto della progettazione minimizza i rischi e le conseguenze avverse di azioni accidentali o non intenzionali.
6. Ridotto sforzo fisico: il prodotto della progettazione può essere usato in modo efficace, confortevole e con sforzo minimo.
7. Dimensione e spazio adatti all'uso e all'approccio: vengono forniti dimensioni e spazi appropriati per l'avvicinamento, la manipolazione e l'uso, indipendentemente dalla corporatura, postura o mobilità dell'utente.

Nel caso dei prodtto interattivi, la progettazione universale rappresenta una sfida difficile per il progettista. Le difficoltà derivano da tre ragioni:

1. Diversità delle tecnlogie disponibili ai diversi utenti. Gli ecosistemi tecnlogici sono in continuo cambiamento. Ad esempio i personal computer, nonnostante la loro elevatissima stnadardizzazione, sono molto diversi fra loro. Ciò pone

enormi problemi di compatibilità fra i sistemi di diversità di prestazioni. Una discriminazione importante riguarda l'accesso alla rete. Gli utenti che dispongono di connessioni a banda larga possono usufruire di servizi che altri utenti non possono utilizzare. Chi dispone di connessioni mobili gode di una flessibilità sconosciuta a chi può accedere alla rete solo da postazioni fisse.

2. Diversità degli individui. Le persone sono molto diverse fra loro. Queste differenze rientrano in tre grandi categorie: fisiche, cognitive, socio-cultarali. Le prime sono relative ai parametri che differenziano gli individui dal punto di vista del loro genere, della struttura corporea e delle loro prestazioni fisiche. Le seconde riguardano le capacità cognitive: memoria, attenzione, attitudine al ragionamento logico o analogica, capacità di ragionamenti, ecc. Le diversità socio-cultarali sono legati agli ambienti scoiali e culturali eni quali i vari individui operano e dai quali vengono formati.
3. Diversità nella capacità d'uso della tecnlogia. Le persone sono diverse nella loro capacità di relazionarsi con i sistemi tecnlogici. Esiste, inoltre, un forte *gap generazionale*, che differenzia la generazione di chi è entrato in contatto con le tecnlogie digitali fin da bambino e le generazioni precedenti.

Dal punto di vista generale ci sono due strade per produrre un design universale. La prima raprpesenta l'approccio tradizionale che consiste nel definire un utente «medio» del sistema, cioè quale tipo di utente al quale il prodotto prioritariamente s'indirizza. Si analizano i suoi bisogni e i suoi contesti d'uso, e si progetta il sistema per lui. Per gli altri utenti si realizzareanno componenti ad hoc, separati dal sistema, i quali lo adatteranno alle varie situazioni d'interesse. Nel caso di utenti con disabilità, queste interfacce speciali sono chiamate *tecnlogie assistive*. Questa è la strada finora maggiormente seguita per la costruzione di sistemi accessibili a utenti con disabilità.
Questo approccio pone diversi problemi. Progettando il sistme per l'utente medio, il progettista non è in grado di tenere conto fin dall'inizio delle necessità degli utenti diversi. Gli adattamenti potranno rivelarsi complessi da realizzare. Un altro tipo di difficoltà è dovuto alla necessità di mantenere la compatibilità fra il sistema e le diverse tecnlogie assistive destinate agli utenti speciali.
A causa di queste difficoltà, ha incominciato a farsi strada una filosofia di progettazione molto diversa. Secondo questo approccio, la progettazione non privilegia l'utente medio, ma tiene in considerazione le necessità di tutte le categorie di utenti. Il sistemi sarà in grado di adattare il proprio funzionamento a ogni specifica classe di utenti, una volta che le informazioni necessarie per questa personalizzazione gli siano satte fornite, per esempio dall'utente stesso in una fase iniziale di configurazione.
I due approcci sono molto diversi: il primo semplifica l'attività di progettazione, che deve considerare una casistica limitata agli utenti «normali». Il secondo richiede che tutti i problemi siano affrontati e risolti in modo sistematico.

Esiste una terza possibilità. In questo caso il sistema, oltre ad essere personalizzabile in fase di configurazione è in grado di monitorare con continuità i comportamenti e le modalità d'uso dell'utente e di addatarvisi modificando il proprio comportamento. In sostanza, il sistema impara durante l'uso. Questi sistemi si chiamano *adattivi* e sono quelli che devono trattare input molto variabili da utente a utente. Questi devono portare a termine una fase di *addestramento iniziale*, per apprendere le caratteristiche comportamentali dello specifico utente. Dopo questa fase, l'addestramento continua durante gli utilizzi successivi, con l'aiuto dell'utente che dovrà correggere il sistema ogni volta che non si comporterà in modo soddisfacente. I tre approcci sono complementari e possono essere compresenti in uno stesso sistema. Possiamo definire l'approccio universale come l'approccio secondo il quale i sistemi sono progettati per essere sufficientemente intelligenti da adattarsi alle richieste o alle modalità di utilizzo dei loro diversi utenti o, quando ciò non è possibile, da permettere un facile interfacciamento con adattatori speciali.

3.6. Livello di maturità della progettazione

Il system-centred design e lo human-centred design non dovrebbero essere considerati due approcci alternativi. Lo human-centred design può essere considerato un approccio più maturo, che contiene le problematiche tecniche del system-centred design, ma le inserisce in un contesto più ampio, che ci permette di comprendere in modo più approfondito le finalità del sistema. Possiamo classificare le attività di progettazione in differenti *livelli di maturità*:

1. Primo livello di maturità: il prodotto funziona. In questo livello, il progettista si occupa principalmente della risoluzione di problemi di natura tecnologica, e si accontenta che le funzioni previste nel sistema siano operative, e non ci siano errori di funzionamento. Questo è il livello più elementare, in cui si accetta di realizzare un sistema anche rudimentale, purchè permetta di eseguire alcuni compiti ritenuti importanti.
2. Secondo livello di maturità: il prodotto fornisce le funzionalità necessarie. A questo livello, il sistema non soltanto funziona, ma realizza tutte le funzionalità ritenute necessarie per gli scopi per cui è concepito. L'attenzione del progettista è posta sulla completezza e sulla qualità delle funzioni del sistema, di cui cura l'affidabilità, le prestazioni, la flessibilità, la modularità. Da parte dell'utente, ci si aspetta che esegua disciplinamente le operazioni specificate nel manuale d'uso.
3. Terzo livello di maturità: il prodotto è facile da usare. Questo è il livello della progettazione human-centred. Non solo il prodotto funziona e offre tutte le funzionalità richieste, ma le organizza in modo adeguato rispetto alle tipologie e alle necessità dei suoi utenti, nei diversi contesti d'uso.
4. Quarto livello di maturità: il prodotto è invisibile durante l'uso. Questo è il livello al quale ogni bravo progettista dovrebbe tendere. In questo caso il prodotto

funziona, fornisce tutte le funzionalità richieste, è usabile e, inoltre, s'integra in modo armonico con i comportamenti del suo utente che questi non si accorge di usarlo.

Ingegneria dell'usabilità

Il termine *ingegneria dell'usabilità* viene usato per denotare la disciplina che studia le tecniche, i metodi e i processi che possono essere utilizzati per progettare e sviluppare sistemi usabili.

> **Definizione 6** *(Ingegneria dell'usabilità)* **.** L'ingegneria dell'usabilità è un processo, basato sull'ingegneria classica, che consiste nello specificare, quantitativamente e in anticipo, quali caratteristiche e in qual misura il prodotto finale da ingegnerizzare dovrà possedere. Questo processo è seguito dall'effettiva realizzazione del prodotto, e dalla dimostrazione che esso effettivamente possiede le caratteristiche pianificate. L'ingegneria non è il processo di costruire un sistema perfetto con risorse infinite. Piuttosto, l'ingegneria è il processo di costruire economicamente un sistema funzionante che soddisfa una necessità. In assenza di specifiche misurabili di usabilità, non c'è alcun modo di determinare le esigenze di usabilità di un prodotto, o di misurare se il prodotto soddisfi o meno tali esigenze. Se non possiamo misurare l'usabilità, non possiamo avere un'ingegneria dell'usabilità.

Inizialmente, l'ingegneria dell'usabilità si è focalizzata sul design dell'interfaccia utente dei sistemi software. Oggi questo termine comprende la totalità delle pratiche utilizzate nel processo di progettazione e sviluppo dei sistemi interattivi, a partire dalla raccolta e analisi iniziale dei requisiti. I principi cardine di questa disciplina si possono riassumere nei seguenti punti:

1. Focalizzazione sull'utente, all'inizio e durante tutto il processo di progettazione;
2. Prove con l'utente durante l'intero processo di progettazione, con analisi qualitative e misure quantitative;
3. Modello di progettazione e sviluppo iterativo, per prototipi successivi.

4.1. Il modello «a cascata»

Quandoa la disciplina dell'ingegneria del software era agli esordi, si pensava che per realizzare un progetto di successo fosse necessario procedere per fasi logiche ben sequenziate, ognuna delle quali ponesse le basi per la fase successiva. Si

partiva dalla raccolta dei requisiti, poi si definivano le specifiche del sistema da realizzare. Si progettava l'intero sistema su carta e lo si codificava nel linguaggio di programmazione scelto. Lo si collaudava a e infine lo si rilasciava. Si passava alla fase successiva solo quando la precedente era completrata e i suoi prodotti vengono approvati formalmente.
Per descrivere questo processo si usa la metafora delle cascata.

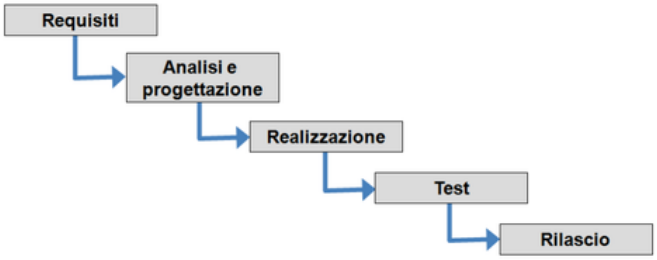


Figura 13: Modello «a cascata»

Ci si accorse ben presto che questo modello non funzionava sempre: nella pratica, in nessun progetto reale, anche se ben gestito, le cose procedevano in maniera semplice e lineare. Si rendeva spesso necessario ritornare sui pacci precedenti, per rivedere e modificare decisioni già prese.
Le cause potevano essere molteplici: il committente richiedeva delle varianti che modificavanoi le specifiche già approvate. Oppure i progettisti scoprivano difficoltà tecniche inattese. Oppure, nella fase di rilascio del sistema, i primi utenti segnalavano delle difficoltà nell'uso che non erano state previste da nessuno e richiedevano cambiamenti consistenti.
Con la maturazione della disciplina dell'ingegneria del software si capì che le cose non potevano funzionare così. Dal punto di vista pratico, è difficile prevedere sulla carta tutti gli aspetti di un sistema complesso.

4.2. Il ciclo compito-artefatto

C'è anche un motivo di natura teorico-concettuale, che fa sì che il modello a cascata non possa funzionare. Questo motivo è racchiuso in un principio generale che possiamo enunciare nel seguentemodo:

o strumento cambia i bisogni del suo utilizzatore e genera nuovi bisogni, che suggeriscono modifiche non previste allo strum

Quindi, per soddisfare le nostre necessità, produciamo strumenti che, a loro volta, generano nuovi bisogni. Costruiamo allora nuovi strumenti, o modifichiamo quelli disponibili, in un ciclo evolutivo infinito, al quale è stato dato il nome di *task-artifact cycle*.



Figura 14: Il ciclo compito-artefatto

Quando definiamo i requisiti di un prodotto che non esiste ancora e che vogliamo realizzare, lo facciamo tenendo conto di determinati bisogni insoddisfatti. Per fare ciò, progettiamo il prodotto ipotizzando degli scenari d'uso che ci sembrano plausibili e realizzando quelle funzioni che, nelle nostre ipotesi, ci sembrano necessarie. Potrà capitare, però, che l'interazione fra utente e prodotto faccia nascere nuovi bisogni. Quindi, non è possibile valutare completamente l'adeguatezza dello strumento ai suoi utenti, prima che questi lo usino effettivamente. Ecco perchè il modello a cascata non può funzionare. Esso prevede che gli utenti siano coinvolti nel processo solo in due momenti: all'inizio, per contribuire a requisiti e specifiche e alla fine, dopo il rilascio. Alla fine, se lo strumento si rivelerà inadeguato sarà troppo tardi per intervenire.

4.3. Modelli iterativi

Data l'inadeguatezza del modello a cascata, ci serve un modello diverso, che coinvolga gli utenti fin da subito, non solo nella stesura di requisiti e specifiche, ma anche, per sperimentare l'uso di versioni preliminari del sistema e aiutarci, con le loro reazioni e indicazioni, a correggere il tiro.
L'idea è di procedere con la realizzazione di una serie di *prototipi*, via via più vicini al sistema finale. Si inizia con un prototipo preliominare, realizzabile a costi ridotti, e lo si sottopone all'utente. Questa prima prova sarà limitata perchè il sistema sarà molto semplificato, però, permette di verificare alcune assunzioni di partenza ed

eventualmente di aggiustare il tiro. Si realizza quindi un nuovo prototipo, sempre incompleto, ma somigliante al sistema finale e lo si sottopone alla prova degli utenti, e così via, per approssimazioni successive, fino alla conclusione del progetto. Quindi, le prove d'uso diventano prate integrante del processo di progettazione.



Figura 15: Modello iterativo

Il primo prototipo sarà rudimentale: in molti casi, soltanto un *mock-up* con il quale effettuare un primo confronto con gli utenti e con il committente. Questo confrontò sarà condotto nella fase di test. In base all'esito del confronto, si apporteranno le necessarie modifiche ai requisiti e al progetto, e si realizzerà un secondo prototipo, più evoluto. All'avanzare del progetto lo sforzo complessivo si sposta progressivamente dalle fasi iniziali del ciclo tradizionale (requisiti e progettazione) alle fasi finali (test e rilascio).
Tutte le attività rappresentate in figura vengono portate avanti «in parallelo» per tutta la durata del progetto, ma l'impegno dedicato a ciascuna di essa cambia nel tempo. Questa situazione è visualizzata nella seguente figura, che mostra per un progetto ipotetico l'andamento nel tempo dell'impegno di risorse sulle singole attività:

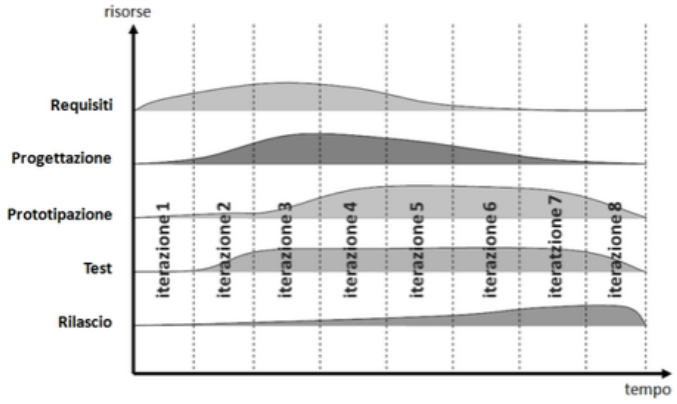


Figura 16: Allocazione delle risorse secondo il modello iterativo

Questo processo è il modello concettualmente corretto per la realizzazione di sistemi complessi.
A fronte di questi vantaggi, esiste il rischio che il processo diverga, a causa delle richieste di modifica che nascono durante le attività di valutazione dei prototipi. Per evitare queste difficoltà, per ogni progetto sarà necessario pianificare il processo iterativo di progettazione e sviluppo in modo che, non sfugga di mano. Inoltre è difficile effettuare una stima dei costi a preventivo e la fluidità delle specifiche rende più difficile comunicazione fra le persone coinvolte (team di progetto, committente).

4.4. Comprendere e specificare il contesto d'uso

Il contesto in cui il sistema sarà utilizzato è definito dalle caratteristiche degli utenti, dei compiti e dell'ambiente fisico e organizzativo. È importante identificare e comprendere i dettagli di questo contesto, per orientare le decisioni iniziali del progetto, e per fornire una base per la loro successiva convalida. La descrizione del contesto d'uso del sistema dovrebbe comporndere i seguenti argomenti:

- Le caratteristiche degli utenti. Le caratteristiche rilevanti potrebbero essere le competenze, le abilità, le esperienze, la formazione, le carateristiche fisiche, le abitudini, le preferenze. Spesso sarà necessario classificare gli utenti in diverse categorie, per esempio in funzione dei diversi ruoli nei confronti del sistema o dei differenti livello di esperienza.
- I compiti che gli utenti dovranno eseguire. Dovrebbero essere anlizzati i compiti che possono influenzare l'usabilità del sistema, per esempio indicandone la frequenza e durata. Si dovranno descrivere eventuali implicazioni riguardanti la salute e alla sicurezza degli utenti.
- L'ambiente nel quale gli utenti utilizzeranno il sistema. Si descriveranno le caratteristiche rilevanti dell'ambiente fisico e sociale, includendo l'ambiente di lavoro, le tecnologie utilizzate, gli eventuali standard adottati, il contesto normativo, la struttura organizzativa e le procedure di lavoor, le abitudini consolidate, ecc.

Tutte queste descrizioni dovranno essere raccolte in un documento di lavoro, che sarà revisionato, corretto e ampliato più volte in accordo con la natura iterativa del processo di progettaszione e sviluppo. Dovranno essere sempre verificate e confermate dagli utenti del sistema.

4.5. Specificare i requisiti utente e organizzativi

Nella maggior parte dei processi di progettaziome, esiste un'attività per la specifica dei requisiti del sistema. Nella progettazione human-centred, quest'attività dovrebbe essere ampliata, per descrivere i requisiti in realzione al contesto d'uso più sopra

specificato.
Si dovrebbero considerare i seguenti aspetti:

- le prestazioni richieste al nuovo sistema in relazione agli obiettivi operativi ed economici;
- i requisiti normativi e legislativi rilevanti, compresi quelli relativi alla sicurezza e alla salute;
- la comunicazione e la cooperazione fra gli utenti e gli attori coinvolti;
- le attività degli utenti (inclusa la ripartizioone dei compiti, il benessere e la motivazione degli utenti);
- la ripartizione dei compiti fra esseri umani e sistemi tecnlogici;
- le prestazioni dei diversi compiti;
- la progettazione delle procedure di lavoro e dell'organizzazione;
- la gestione del cambiamento, incluse le attività di addestramento e il personale coinvolto;
- la fattibilità delle diverse operazioni, comprese quelle di manutenzione;
- la progettazione dei posti di lavoro e l'interfaccia uomo-computer.

I requisiti dovrebbero essere organizzati per livello di priorità, e formulati in modo da permettere la loro successiva convalida mediante opportuni test. Dovrebbero essere confermati o aggiornati lungo tutta la durata del progetto.

4.6. Produrre soluzioni di progetto

In questa fase si individuano le possibili soluzioni di progetto, basandosi sullo stato dell'arte, sulle conoscenze ed esperienze dei partecipanti e sui risultati dell'analisi del contesto d'uso. Lo standard identifica le seguenti attività:

- Utilizzare le conoscenze disponibili per sviluppare proposte di progetto con un approccio multi-disciplinare. Molte organizzazioni dispongono di linee guida per l'interfaccia utente, conoscenze sul prodotto e sul mercato che possono essere utili nelle fasi inziiali del progetto. Inoltre, esistono linee guida e standard per l'ergonomia e i fattori umani, elaborati dagli enti di standardizzazione.
- Rendere le soluzioni di progetto più concrete, utilizzando simulazioni, modelli e prototipi di vario tipo. L'uso di prototipi permette ai progettisti di comunicare più efficacemente con gli utentei, e riduce la necessità di costosi rifacimenti, che possono essere necessari quando il prodotto viene sottoposto a revisione soltanto più tardi nel ciclo di vita.
- Presentare le soluzioni di progetto agli utenti, permettendo loro di eseguire i compiti che il sistema è destinato a supportare. Gli utenti possono essere coinvolti molto presto nel progetto. È possibile presentare agli utenti le bozze delle schermate o una rappresentazione del prodotto, chiedendo loro di provarli in un

contesto realistico. In tal modo si possono valutare rapidamente ed economicamente aspetti del progetto.

- Modificare il progetto in conseguenza delle reazioni degli utenti, e ripetere questo proceso fino a che gli obiettivi della progettazione non siano raggiunti. I commenti dell'utente, o le difficoltà osservate durante l'utilizzo del prototipo, suggeriranno modifiche al progetto, per migliorarne l'usabilità. Questi feedback potranno anche essere di aiuto per raffinare gli obiettivi complessivi del sistema.
- Gestire l'iterazione delle soluzioni di progetto. Per controllare i progressi della progettazione iterativa, si dovrebbero registrare i risultati delle attività precedenti. Questa documentazione potrà essere esclusivamente descrittiva. La documentazione descriverà lo scopo dei vari prototipi, le modalità operative del loro utilizzo e i problemi individuati, con le conseguenti modifiche del progetto.

4.7. Valutare il progetto nei confronti dei requisiti.

La valutazione è un passo essenziale in una progettazione human-centred, e dovrebbe essere compiuta in tutte le fasi del ciclo di vita del sistema. Lo standard identifica le seguenti attività:

- Produrre il piano di valutazione Il processo di valutazioen dovrebbe essere pianificato, precisando, quali parti del sistema devono essere valutati e come; quali prototipi dovranno essere realizzati e come deve essere eseguita la valutazione e con quali risorse; quali dovranno essere le interazioni con gli utenti e come dovrà essere condotta l'analisi dei risultati. Le tecniche di valutazione variano secondo i casi e la scelta è determinata dalla natura del sistema, dai vincoli economici e di tempo, e dalla fase del ciclo di sviluppo in cui si svolge la valutazione.
- Fornire feedback per la progettazione Per influenzare la progettazione, la valutazione dovrebbe essere condotta in ogni fase del ciclo di vita del sistema. La valutazione può essere veloce ed economica, e permette di identificare i problemi maggiori, ma non basta a garantire il successo di un sistema interattivo.
- Verificare se gli obiettivi sono stati raggiunti. La valutazione può essere usata per dimostrare che un particolare progetto soddisfa i requisiti human-centred, oppure per verificare la conformità a standard internazionali, nazionali, locali, aziendali o legali. Per ottenere risultati validi, la valutazione dovrebbe utilizzare metodi appropriati, con un campione rappresentativo di utenti che eseguono compiti realistici.
- Validazione sul campo Lo scopo delle validazione sul campo è provare il funzionamento del sistema finale durante l'uso effettivo, per assicurare che esso soddisfi i requisiti degli utenti, dei compiti e dell'ambiente. Per fare ciò si possono analizzare dati raccolti dall'help desk, rapporti dal campo, feedback da utenti reali, dati prestazionali, rapporti sull'impatto sulla salute, richieste di miglioramenti e modifiche da parte di utenti.

- Monitoraggio di lungo termine Dovrebbe esiste un processo per il monitoraggio di lungo termine dell'uso del prodotto o del sistema, che consiste nel raccogliere input dagli utenti, con modalità differenti, lungo un certo periodo di tempo.
- Documentazione dei risultati Allo scopo di gestire il processo di progettazione iterativo, i risultati delle valutazioni dovrebbero essere registrati in modo sistematico. Dovrebbe esistere un'adeguata evidenza che un numero adeguato di utenti abbia partecipato al test e che questi utenti siano rappresentativi delle categorie identificate nei requisiti, che i test siano adeguati a fornire indicazioni attendibili e, infine, che siano stati usati metodi appropriati per il test le la raccolta dei dati.

4.8. UCD Sprint

L'HCD è spesso anche chiamato «User Centred Design (UCD)». Lo UCD Sprint è un processo **passo-dopo-passo** progettato per favorire la generazione costruttiva di idee all'interno del team software. Presenta una struttura chiara con passaggi definiti per ogni parte del processo di progettazione. La metodologia è accessibile a team meno esperti, rendendola adatta alla formazione nel design. Il processo UCD Sprint è stato sviluppato attraverso corsi intensivi di interaction design da ricercatori leader nell'interazione uomo-computer. Per i professionisti che hanno familiarità con i metodi di user-centred desigm, il processo può essere applicato più rapidamente.
Lo User-Centred Design Sprint è suddiviso in 3 parti. Ogni parte è divisa in processi distinti, pensati per guidare i partecipanti attraverso:

- Un'analisi degli utenti.
- Una sessione di brainstoring e schizzi.
- Un prototipo finale, testing e analisi dei risultati.

Lo UCD Sprint è un'istanz del modello HCD dell'ISO 9241:210. Come tale, ne ha tutte le caratteristiche. In particolare, l'UCD Sprint è un processo ciclico, e i progettisti possono ripetere le varie fasi liberamente in base alle necessità di progetto.

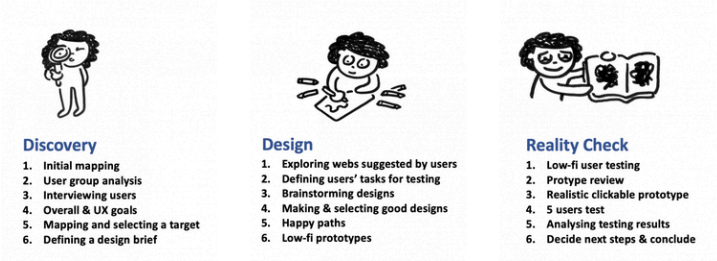


Figura 17: Fasi dell'UCD Sprint

4.8.1. Design Thinking

Il Design Thinkign è un'**ideologia supportata da un processo correlato**. Una definizione completa richiede la comprensione di entrambi.

Definizione 7 *(Design thinking)* **.** L'ideologia del *design thinking* afferma che un approccio pratico e incentrato sull'utente alla risoluzione dei problemi può portare all'innovazione e l'innovazione può portare alla differenziazione e a un vantaggio competitivo. Questo approccio pratico e incentrato sull'utente è definito dal processo del design thinking e comprende 6 fasi distinte, come definito e illustrato di seguito.



Figura 18: Fasi del design thinking

Il Design Thinking è incentrato sull'utente che parte dai dati dell'utente:

- crea artefatti di design che rispondono a esigenze utente reali e non immaginarie
- testa tali artefatti con utenti reali.

Questo processo sfrutta l'esperienza collettiva e stabilisce un linguaggio comune, oltre a un consenso all'interno del team. Incoraggia l'innovazione esplorando molteplici soluzioni per lo stesso problema. Jakob Nielsen afferma che «un'interfaccia meravigliosa che risolve il problema sbagliato fallirà».
Anaizziamo nel dettaglio le fasi del Design Thinking:

1. **Empatizzare**. In questa fase si conducono ricerche al fine di sviluppare la conoscenza su ciò che i tuoi utenti fanno, dicono, pensano e provano.

2. **Definire**. Combinare tutte le ricerche e osservare dove esistono i problemi dei tuoi utenti. Mentre identifichi le esigenze dei tuoi utenti, inizia a evidenziare le opportunità di innovazione.
3. **Ideare**. Fare brainstorming su una serie di idee folli e creative che rispondano alle esigenze degli utenti insoddisfatte identificate nella fase di definizione.
4. **Prototipare**. Costruire rappresentazioni reali e tangibili per un sottoinsieme delle tue idee. L'obiettivi di questa fase è capire quali componenti delle tue idee funzionano, e quali no.
5. **Testare**. Tornare dai tuoi utenti per ricevere feedback. Bisogna chiedersi se tale soluzione soddisfa le esigenze degli utenti.
6. **Implementare**. Mettere in pratica la visione. Assicurarsi che la soluzione sia materializzata e influenzi la vita dei tuoi utenti finali.

Il Design Thinking è un processo iterativo, come mostrato nella seguente figura.

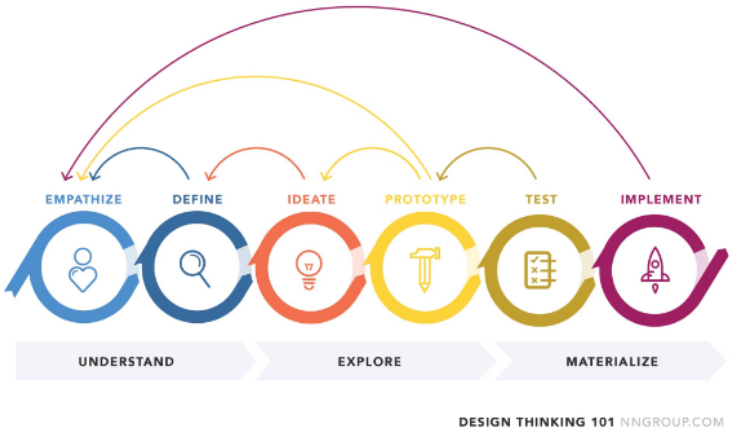


Figura 19: Iteratività del design thinking

UCD Sprint e Design Thinking sembrano simili proprio perchè sono istanze dello stesso modello HCD. Quindi condividono molti aspetti. La scelta di un processo piuttosto che un altro è spesso legato alle esigenze dei progettisti e alle esigenze dei gruppi di lavoro.